

MyFlowSim

1.0

Generated by Doxygen 1.17.0

1 Directory Hierarchy	1
1.1 Directories	1
2 Hierarchical Index	3
2.1 Class Hierarchy	3
3 Class Index	5
3.1 Class List	5
4 File Index	7
4.1 File List	7
5 Directory Documentation	9
5.1 C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/test/functional Directory Reference	9
5.2 C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/include Directory Reference	10
5.3 C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/src Directory Reference	10
5.4 C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/test Directory Reference	11
5.5 C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/test/unit Directory Reference	11
6 Class Documentation	13
6.1 Flow Class Reference	13
6.1.1 Detailed Description	16
6.1.2 Constructor & Destructor Documentation	17
6.1.2.1 Flow() [1/3]	17
6.1.2.2 Flow() [2/3]	17
6.1.2.3 ~Flow()	18
6.1.2.4 Flow() [3/3]	18
6.1.3 Member Function Documentation	19
6.1.3.1 execute()	19
6.1.3.2 getName()	19
6.1.3.3 getSource()	19
6.1.3.4 getTarget()	20
6.1.3.5 operator=()	20
6.1.3.6 setName()	22
6.1.3.7 setSource()	22
6.1.3.8 setTarget()	22
6.1.4 Member Data Documentation	22
6.1.4.1 name	22
6.1.4.2 source	22
6.1.4.3 target	23

6.2 FlowComplejo Class Reference	23
6.2.1 Detailed Description	26
6.2.2 Constructor & Destructor Documentation	27
6.2.2.1 FlowComplejo() [1/3]	27
6.2.2.2 FlowComplejo() [2/3]	27
6.2.2.3 FlowComplejo() [3/3]	28
6.2.2.4 ~FlowComplejo()	28
6.2.3 Member Function Documentation	28
6.2.3.1 execute()	28
6.2.3.2 operator=()	29
6.3 FlowExponencial Class Reference	30
6.3.1 Detailed Description	32
6.3.2 Constructor & Destructor Documentation	33
6.3.2.1 FlowExponencial() [1/3]	33
6.3.2.2 FlowExponencial() [2/3]	33
6.3.2.3 FlowExponencial() [3/3]	34
6.3.2.4 ~FlowExponencial()	34
6.3.3 Member Function Documentation	34
6.3.3.1 execute()	34
6.3.3.2 operator=()	35
6.4 FlowLogistico Class Reference	36
6.4.1 Detailed Description	38
6.4.2 Constructor & Destructor Documentation	39
6.4.2.1 FlowLogistico() [1/3]	39
6.4.2.2 FlowLogistico() [2/3]	39
6.4.2.3 FlowLogistico() [3/3]	40
6.4.2.4 ~FlowLogistico()	40
6.4.3 Member Function Documentation	40
6.4.3.1 execute()	40
6.4.3.2 operator=()	41
6.5 Model Class Reference	42
6.5.1 Detailed Description	43
6.5.2 Constructor & Destructor Documentation	43
6.5.2.1 Model() [1/2]	43
6.5.2.2 ~Model()	43
6.5.2.3 Model() [2/2]	44
6.5.3 Member Function Documentation	44
6.5.3.1 add() [1/2]	44
6.5.3.2 add() [2/2]	45
6.5.3.3 operator=()	45
6.5.3.4 run()	46
6.5.3.5 showModel()	47

6.5.4 Member Data Documentation	48
6.5.4.1 flows	48
6.5.4.2 systems	48
6.6 System Class Reference	48
6.6.1 Detailed Description	50
6.6.2 Constructor & Destructor Documentation	50
6.6.2.1 System() [1/3]	50
6.6.2.2 System() [2/3]	51
6.6.2.3 ~System()	51
6.6.2.4 System() [3/3]	51
6.6.3 Member Function Documentation	52
6.6.3.1 getName()	52
6.6.3.2 getValue()	52
6.6.3.3 operator=()	53
6.6.3.4 setName()	53
6.6.3.5 setValue()	54
6.6.4 Member Data Documentation	54
6.6.4.1 name	54
6.6.4.2 value	54
7 File Documentation	55
7.1 C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/include/flow.h File Reference	55
7.2 flow.h	56
7.3 C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/include/flows.h File Reference	57
7.4 flows.h	58
7.5 C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/include/↔ functional_test.h File Reference	59
7.5.1 Function Documentation	60
7.5.1.1 complexFuncionalTest()	60
7.5.1.2 exponentialFuncionalTest()	61
7.5.1.3 logisticalFuncionalTest()	61
7.6 functional_test.h	62
7.7 C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/include/model.h File Reference	63
7.8 model.h	63
7.9 C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/include/system.h File Reference	64
7.10 system.h	65
7.11 C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/src/flow.cpp File Reference	65
7.11.1 Detailed Description	66

7.12	C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/src/main.cpp	
	File Reference	66
7.12.1	Detailed Description	67
7.12.2	Macro Definition Documentation	67
	7.12.2.1 MAIN	67
7.12.3	Function Documentation	67
	7.12.3.1 main()	67
7.13	C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/test/functional/↔	
	main.cpp File Reference	68
7.13.1	Detailed Description	68
7.13.2	Macro Definition Documentation	68
	7.13.2.1 MAIN_FUNCIONAL_TESTS	68
7.13.3	Function Documentation	69
	7.13.3.1 main()	69
7.14	C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/test/unit/↔	
	main.cpp File Reference	69
7.14.1	Function Documentation	69
	7.14.1.1 main()	69
7.15	C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/src/model.cpp	
	File Reference	70
7.15.1	Detailed Description	70
7.16	C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/src/system.cpp	
	File Reference	70
7.16.1	Detailed Description	71
7.17	C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/test/functional/↔	
	flows.cpp File Reference	71
7.17.1	Detailed Description	72
7.18	C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/test/functional/↔	
	functional_test.cpp File Reference	72
7.18.1	Function Documentation	73
	7.18.1.1 complexFuncionalTest()	73
	7.18.1.2 exponentialFuncionalTest()	74
	7.18.1.3 floatingPointComparison()	75
	7.18.1.4 logisticalFuncionalTest()	76
7.19	C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/test/unit/unit_↔	
	test.cpp File Reference	77
Index		79

Chapter 1

Directory Hierarchy

1.1 Directories

functional	9
flows.cpp	71
functional_test.cpp	72
main.cpp	68
include	10
flow.h	55
flows.h	57
functional_test.h	59
model.h	63
system.h	64
src	10
flow.cpp	65
main.cpp	66
model.cpp	70
system.cpp	70
test	11
functional	9
flows.cpp	71
functional_test.cpp	72
main.cpp	68
unit	11
main.cpp	69
unit_test.cpp	77
unit	11
main.cpp	69
unit_test.cpp	77

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Flow	13
FlowComplexo	23
FlowExponencial	30
FlowLogistico	36
Model	42
System	48

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Flow	Classe abstrata responsável por representar um fluxo	13
FlowComplexo	Classe responsável pelo fluxo complexo	23
FlowExponencial	Classe responsável pelo fluxo exponencial	30
FlowLogistico	Classe responsável pelo fluxo logístico	36
Model	Classe responsável por representar um modelo de simulação	42
System	Classe responsável por representar um sistema	48

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

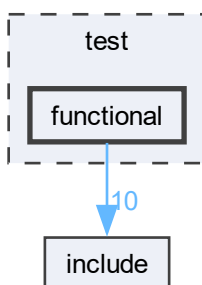
C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/include/flow.h	55
C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/include/flows.h . . .	57
C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/include/functional_test.h 59	
C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/include/model.h . .	63
C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/include/system.h . .	64
C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/src/flow.cpp Implementação da classe abstrata Flow	65
C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/src/main.cpp Arquivo principal do simulador	66
C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/src/model.cpp Implementação da classe Model	70
C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/src/system.cpp Implementação da classe System	70
C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/test/functional/↵ flows.cpp Implementação dos fluxos do simulador	71
C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/test/functional/↵ functional_test.cpp	72
C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/test/functional/↵ main.cpp Arquivo principal dos testes funcionais	68
C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/test/unit/main.cpp .	69
C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/test/unit/unit_test.cpp 77	

Chapter 5

Directory Documentation

5.1 C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/test/functional Directory Reference

Directory dependency graph for functional:



Files

- file [flows.cpp](#)
Implementação dos fluxos do simulador.
- file [functional_test.cpp](#)
- file [main.cpp](#)
Arquivo principal dos testes funcionais.

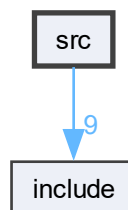
5.2 C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/include Directory Reference

Files

- file [flow.h](#)
- file [flows.h](#)
- file [functional_test.h](#)
- file [model.h](#)
- file [system.h](#)

5.3 C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/src Directory Reference

Directory dependency graph for src:

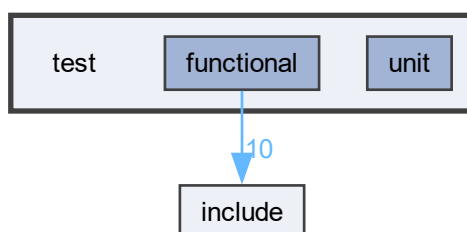


Files

- file [flow.cpp](#)
Implementação da classe abstrata [Flow](#).
- file [main.cpp](#)
Arquivo principal do simulador.
- file [model.cpp](#)
Implementação da classe [Model](#).
- file [system.cpp](#)
Implementação da classe [System](#).

5.4 C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/test Directory Reference

Directory dependency graph for test:

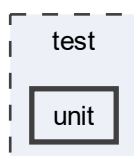


Directories

- directory [functional](#)
- directory [unit](#)

5.5 C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/test/unit Directory Reference

Directory dependency graph for unit:



Files

- file [main.cpp](#)
- file [unit_test.cpp](#)

Chapter 6

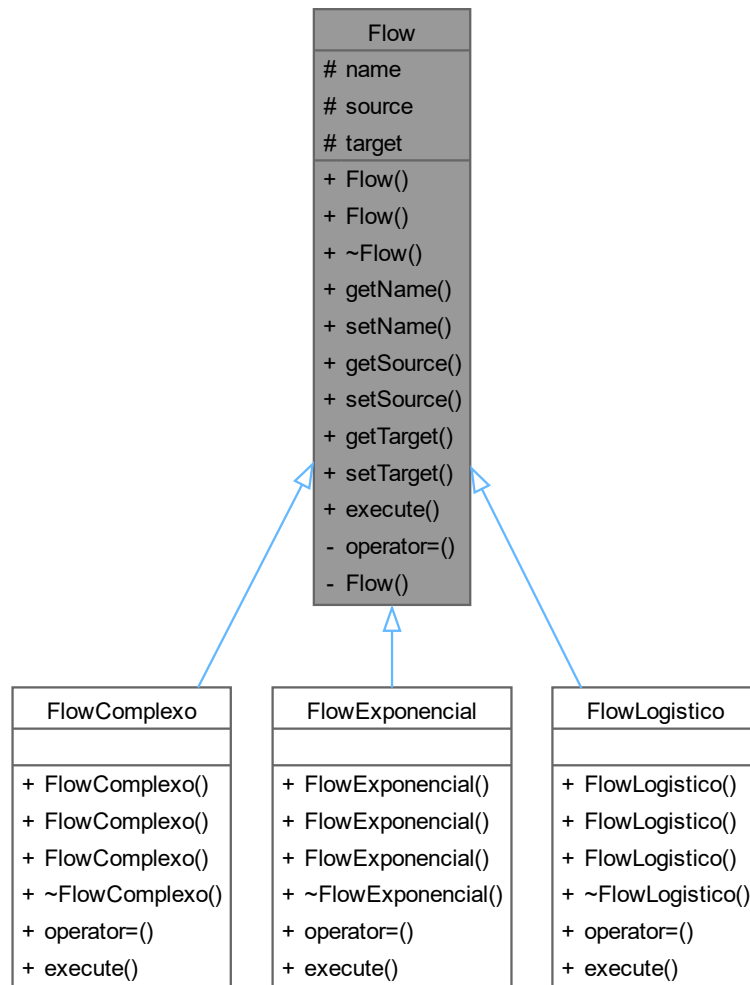
Class Documentation

6.1 Flow Class Reference

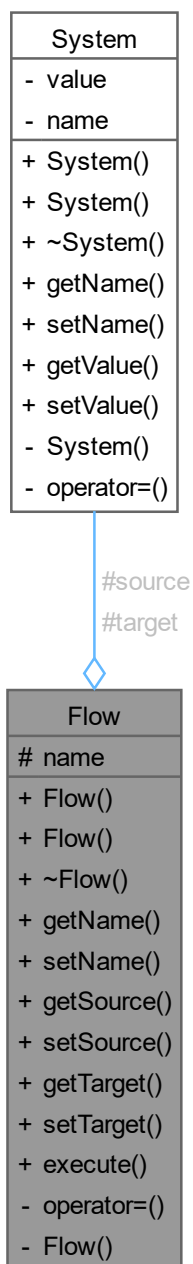
Classe abstrata responsável por representar um fluxo.

```
#include <flow.h>
```

Inheritance diagram for Flow:



Collaboration diagram for Flow:



Public Member Functions

- `Flow ()`
Construtor padrão da classe Flow.
- `Flow (const string &name, System *source, System *target)`
Construtor parametrizado da classe Flow.
- `virtual ~Flow ()`

Destrutor virtual da classe [Flow](#).

- [string getName \(\)](#) const
Retorna o nome do fluxo.
- [void setName \(const string &name\)](#)
Define o nome do fluxo.
- [System * getSource \(\)](#) const
Retorna o sistema de origem.
- [void setSource \(System *source\)](#)
Define o sistema de origem.
- [System * getTarget \(\)](#) const
Retorna o sistema de destino.
- [void setTarget \(System *target\)](#)
Define o sistema de destino.
- [virtual double execute \(\)=0](#)
Executa o cálculo do fluxo.

Protected Attributes

- [string name](#)
Nome do fluxo.
- [System * source](#)
Sistema de origem do fluxo.
- [System * target](#)
Sistema de destino do fluxo.

Private Member Functions

- [Flow & operator= \(const Flow &other\)](#)
Operador de atribuição privado.
- [Flow \(const Flow &other\)](#)
Construtor de cópia privado.

6.1.1 Detailed Description

Classe abstrata responsável por representar um fluxo.

A classe [Flow](#) define a estrutura básica de um fluxo em um modelo de dinâmica de sistemas, conectando um sistema de origem a um sistema de destino.

6.1.2 Constructor & Destructor Documentation

6.1.2.1 Flow() [1/3]

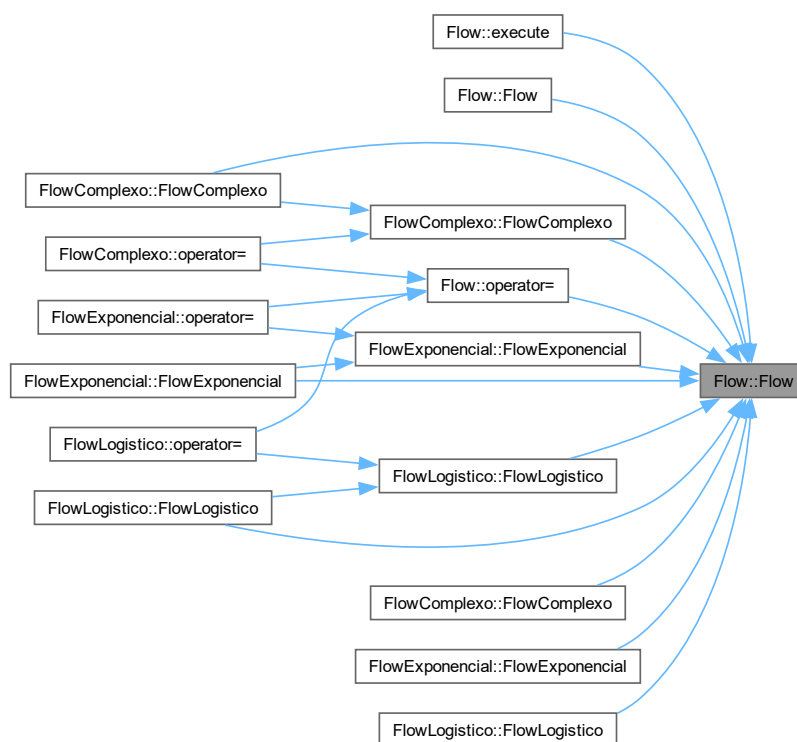
```
Flow::Flow ()
```

Construtor padrão da classe [Flow](#).

Inicializa:

- nome vazio
- source nulo
- target nulo

Here is the caller graph for this function:



6.1.2.2 Flow() [2/3]

```
Flow::Flow (
    const string & name,
    System * source,
    System * target)
```

Construtor parametrizado da classe [Flow](#).

Parameters

<i>name</i>	Nome do fluxo.
<i>source</i>	Sistema de origem.
<i>target</i>	Sistema de destino.

6.1.2.3 ~Flow()

```
Flow::~~Flow () [virtual]
```

Destrutor virtual da classe [Flow](#).

Destrutor da classe [Flow](#).

6.1.2.4 Flow() [3/3]

```
Flow::Flow (  
    const Flow & other) [private]
```

Construtor de cópia privado.

Construtor de cópia da classe [Flow](#).

Impede cópia da classe [Flow](#).

Parameters

<i>other</i>	Outro objeto Flow .
<i>other</i>	Outro objeto Flow .

Here is the call graph for this function:



6.1.3 Member Function Documentation

6.1.3.1 execute()

```
virtual double Flow::execute () [pure virtual]
```

Executa o cálculo do fluxo.

Método abstrato implementado pelas subclasses.

Returns

Valor calculado pelo fluxo.

Implemented in [FlowComplexo](#), [FlowExponencial](#), and [FlowLogistico](#).

Here is the call graph for this function:



6.1.3.2 getName()

```
string Flow::getName () const
```

Retorna o nome do fluxo.

Returns

Nome do fluxo.

6.1.3.3 getSource()

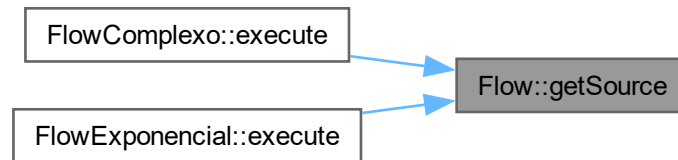
```
System * Flow::getSource () const
```

Retorna o sistema de origem.

Returns

Ponteiro para o sistema de origem.

Here is the caller graph for this function:

**6.1.3.4 getTarget()**

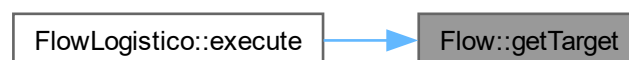
```
System * Flow::getTarget () const
```

Retorna o sistema de destino.

Returns

Ponteiro para o sistema de destino.

Here is the caller graph for this function:

**6.1.3.5 operator=()**

```
Flow & Flow::operator= (  
    const Flow & other) [private]
```

Operador de atribuição privado.

Operador de atribuição da classe [Flow](#).

Impede atribuição entre objetos [Flow](#).

Parameters

<i>other</i>	Outro objeto Flow .
--------------	-------------------------------------

Returns

Referência para o objeto atual.

Copia os atributos de outro fluxo.

Parameters

<i>other</i>	Outro objeto Flow .
--------------	-------------------------------------

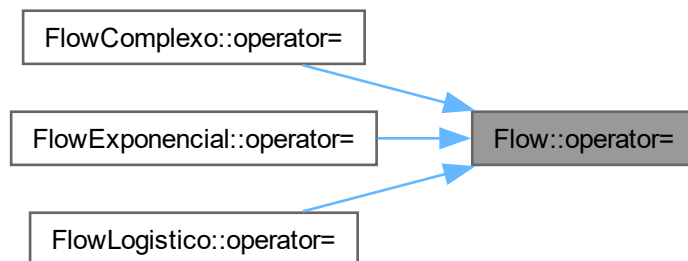
Returns

Referência para o objeto atual.

Here is the call graph for this function:



Here is the caller graph for this function:



6.1.3.6 setName()

```
void Flow::setName (
    const string & name)
```

Define o nome do fluxo.

Parameters

<i>name</i>	Novo nome do fluxo.
-------------	---------------------

6.1.3.7 setSource()

```
void Flow::setSource (
    System * source)
```

Define o sistema de origem.

Parameters

<i>source</i>	Novo sistema de origem.
---------------	-------------------------

6.1.3.8 setTarget()

```
void Flow::setTarget (
    System * target)
```

Define o sistema de destino.

Parameters

<i>target</i>	Novo sistema de destino.
---------------	--------------------------

6.1.4 Member Data Documentation

6.1.4.1 name

```
string Flow::name [protected]
```

Nome do fluxo.

6.1.4.2 source

```
System* Flow::source [protected]
```

Sistema de origem do fluxo.

6.1.4.3 target

```
System* Flow::target [protected]
```

Sistema de destino do fluxo.

The documentation for this class was generated from the following files:

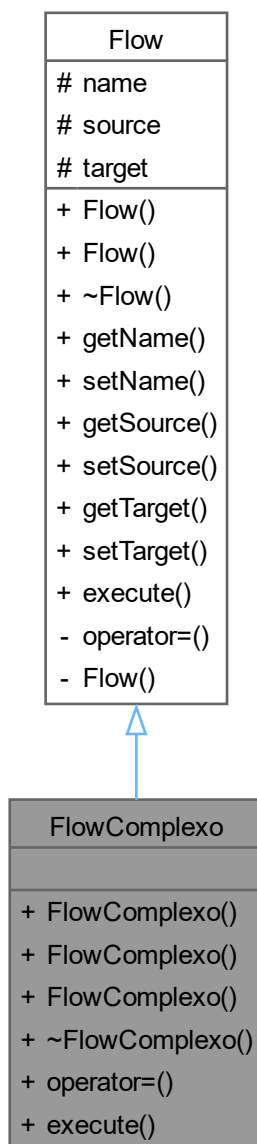
- C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/include/[flow.h](#)
- C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/src/[flow.cpp](#)

6.2 FlowComplexo Class Reference

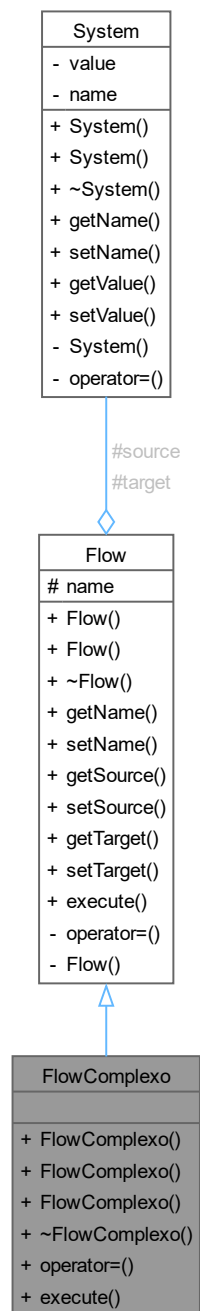
Classe responsável pelo fluxo complexo.

```
#include <flows.h>
```

Inheritance diagram for FlowComplexo:



Collaboration diagram for FlowComplexo:



Public Member Functions

- [FlowComplexo](#) ()
Construtor padrão da classe [FlowComplexo](#).
- [FlowComplexo](#) (const string &name, [System](#) *source, [System](#) *target)
Construtor parametrizado da classe [FlowComplexo](#).
- [FlowComplexo](#) (const [FlowComplexo](#) &other)

- Construtor de cópia da classe [FlowComplexo](#).*
- virtual [~FlowComplexo](#) ()
Destrutor virtual da classe [FlowComplexo](#).
- [FlowComplexo](#) & [operator=](#) (const [FlowComplexo](#) &other)
Operador de atribuição da classe [FlowComplexo](#).
- double [execute](#) () override
Executa o cálculo do fluxo complexo.

Public Member Functions inherited from [Flow](#)

- [Flow](#) ()
Construtor padrão da classe [Flow](#).
- [Flow](#) (const string &name, [System](#) *source, [System](#) *target)
Construtor parametrizado da classe [Flow](#).
- virtual [~Flow](#) ()
Destrutor virtual da classe [Flow](#).
- string [getName](#) () const
Retorna o nome do fluxo.
- void [setName](#) (const string &name)
Define o nome do fluxo.
- [System](#) * [getSource](#) () const
Retorna o sistema de origem.
- void [setSource](#) ([System](#) *source)
Define o sistema de origem.
- [System](#) * [getTarget](#) () const
Retorna o sistema de destino.
- void [setTarget](#) ([System](#) *target)
Define o sistema de destino.

Additional Inherited Members

Protected Attributes inherited from [Flow](#)

- string [name](#)
Nome do fluxo.
- [System](#) * [source](#)
Sistema de origem do fluxo.
- [System](#) * [target](#)
Sistema de destino do fluxo.

6.2.1 Detailed Description

Classe responsável pelo fluxo complexo.

Implementa um fluxo com múltiplas interações entre sistemas.

6.2.2 Constructor & Destructor Documentation

6.2.2.1 FlowComplexo() [1/3]

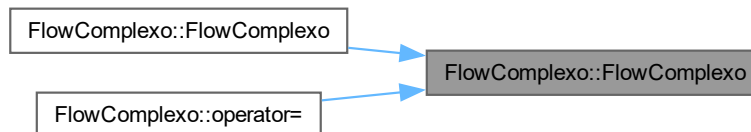
```
FlowComplexo::FlowComplexo ()
```

Construtor padrão da classe [FlowComplexo](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.2.2.2 FlowComplexo() [2/3]

```
FlowComplexo::FlowComplexo (  
    const string & name,  
    System * source,  
    System * target)
```

Construtor parametrizado da classe [FlowComplexo](#).

Parameters

<i>name</i>	Nome do fluxo.
<i>source</i>	Sistema de origem.
<i>target</i>	Sistema de destino.

Here is the call graph for this function:



6.2.2.3 FlowComplexo() [3/3]

```
FlowComplexo::FlowComplexo (
    const FlowComplexo & other)
```

Construtor de cópia da classe [FlowComplexo](#).

Parameters

<i>other</i>	Outro objeto FlowComplexo .
--------------	---

Here is the call graph for this function:



6.2.2.4 ~FlowComplexo()

```
FlowComplexo::~~FlowComplexo () [virtual]
```

Destrutor virtual da classe [FlowComplexo](#).

Destrutor da classe [FlowComplexo](#).

6.2.3 Member Function Documentation

6.2.3.1 execute()

```
double FlowComplexo::execute () [override], [virtual]
```

Executa o cálculo do fluxo complexo.

Returns

Valor calculado pelo fluxo.

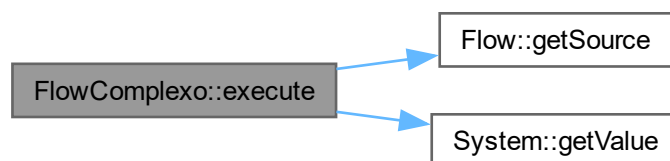
Calcula a transferência entre sistemas utilizando o modelo complexo.

Returns

Valor calculado do fluxo.

Implements [Flow](#).

Here is the call graph for this function:

**6.2.3.2 operator=()**

```
FlowComplexo & FlowComplexo::operator= (
    const FlowComplexo & other)
```

Operador de atribuição da classe [FlowComplexo](#).

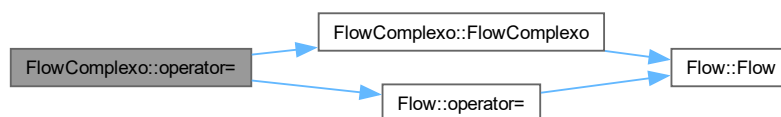
Parameters

<i>other</i>	Outro objeto FlowComplexo .
--------------	---

Returns

Referência para o objeto atual.

Here is the call graph for this function:



The documentation for this class was generated from the following files:

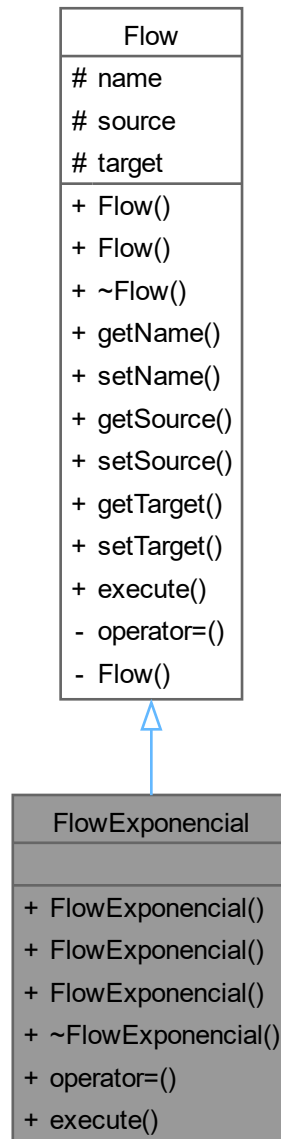
- `C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/include/flows.h`
- `C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/test/functional/flows.cpp`

6.3 FlowExponencial Class Reference

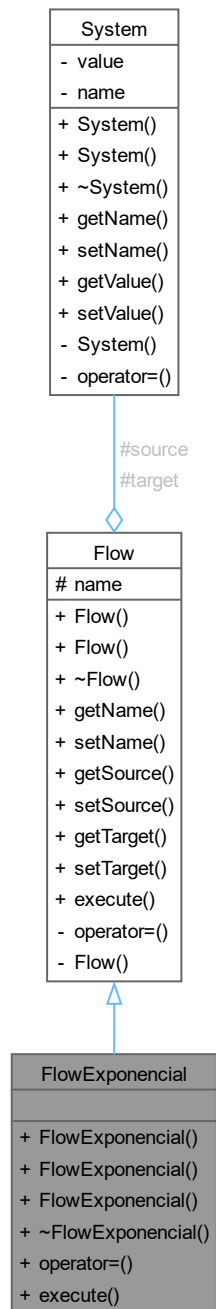
Classe responsável pelo fluxo exponencial.

```
#include <flows.h>
```

Inheritance diagram for FlowExponencial:



Collaboration diagram for FlowExponencial:



Public Member Functions

- [FlowExponencial](#) ()
Construtor padrão da classe [FlowExponencial](#).
- [FlowExponencial](#) (const string &name, [System](#) *source, [System](#) *target)
Construtor parametrizado da classe [FlowExponencial](#).
- [FlowExponencial](#) (const [FlowExponencial](#) &other)

- Construtor de cópia da classe [FlowExponencial](#).*
- virtual [~FlowExponencial](#) ()
Destrutor virtual da classe [FlowExponencial](#).
- [FlowExponencial](#) & [operator=](#) (const [FlowExponencial](#) &other)
Operador de atribuição da classe [FlowExponencial](#).
- double [execute](#) () override
Executa o cálculo do fluxo exponencial.

Public Member Functions inherited from [Flow](#)

- [Flow](#) ()
Construtor padrão da classe [Flow](#).
- [Flow](#) (const string &name, [System](#) *source, [System](#) *target)
Construtor parametrizado da classe [Flow](#).
- virtual [~Flow](#) ()
Destrutor virtual da classe [Flow](#).
- string [getName](#) () const
Retorna o nome do fluxo.
- void [setName](#) (const string &name)
Define o nome do fluxo.
- [System](#) * [getSource](#) () const
Retorna o sistema de origem.
- void [setSource](#) ([System](#) *source)
Define o sistema de origem.
- [System](#) * [getTarget](#) () const
Retorna o sistema de destino.
- void [setTarget](#) ([System](#) *target)
Define o sistema de destino.

Additional Inherited Members

Protected Attributes inherited from [Flow](#)

- string [name](#)
Nome do fluxo.
- [System](#) * [source](#)
Sistema de origem do fluxo.
- [System](#) * [target](#)
Sistema de destino do fluxo.

6.3.1 Detailed Description

Classe responsável pelo fluxo exponencial.

Implementa um fluxo baseado em crescimento exponencial entre sistemas.

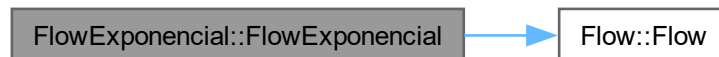
6.3.2 Constructor & Destructor Documentation

6.3.2.1 FlowExponencial() [1/3]

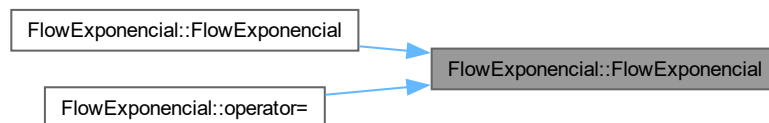
`FlowExponencial::FlowExponencial ()`

Construtor padrão da classe [FlowExponencial](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.3.2.2 FlowExponencial() [2/3]

```

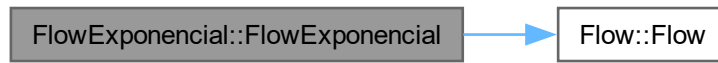
FlowExponencial::FlowExponencial (
    const string & name,
    System * source,
    System * target)
  
```

Construtor parametrizado da classe [FlowExponencial](#).

Parameters

<i>name</i>	Nome do fluxo.
<i>source</i>	Sistema de origem.
<i>target</i>	Sistema de destino.

Here is the call graph for this function:



6.3.2.3 FlowExponencial() [3/3]

```
FlowExponencial::FlowExponencial (
    const FlowExponencial & other)
```

Construtor de cópia da classe [FlowExponencial](#).

Parameters

<i>other</i>	Outro objeto FlowExponencial .
--------------	--

Here is the call graph for this function:



6.3.2.4 ~FlowExponencial()

```
FlowExponencial::~~FlowExponencial () [virtual]
```

Destrutor virtual da classe [FlowExponencial](#).

Destrutor da classe [FlowExponencial](#).

6.3.3 Member Function Documentation

6.3.3.1 execute()

```
double FlowExponencial::execute () [override], [virtual]
```

Executa o cálculo do fluxo exponencial.

Returns

Valor calculado pelo fluxo.

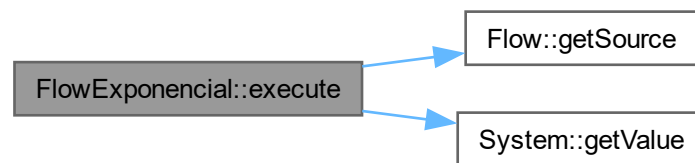
O valor transferido corresponde a 1% do valor do sistema de origem.

Returns

Valor calculado do fluxo.

Implements [Flow](#).

Here is the call graph for this function:

**6.3.3.2 operator=()**

```
FlowExponencial & FlowExponencial::operator= (
    const FlowExponencial & other)
```

Operador de atribuição da classe [FlowExponencial](#).

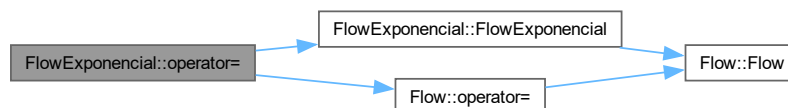
Parameters

<i>other</i>	Outro objeto FlowExponencial .
--------------	--

Returns

Referência para o objeto atual.

Here is the call graph for this function:



The documentation for this class was generated from the following files:

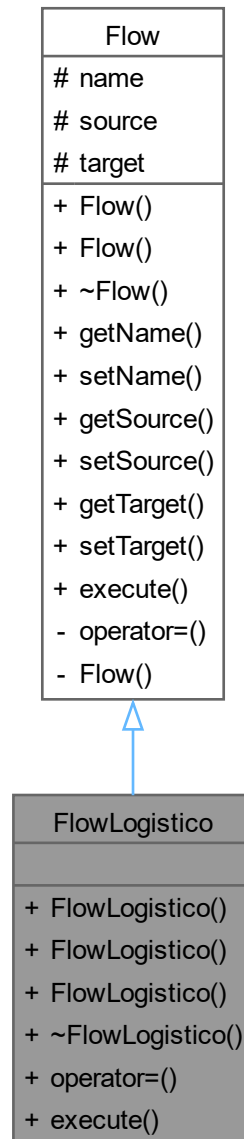
- `C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/include/flows.h`
- `C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/test/functional/flows.cpp`

6.4 FlowLogistico Class Reference

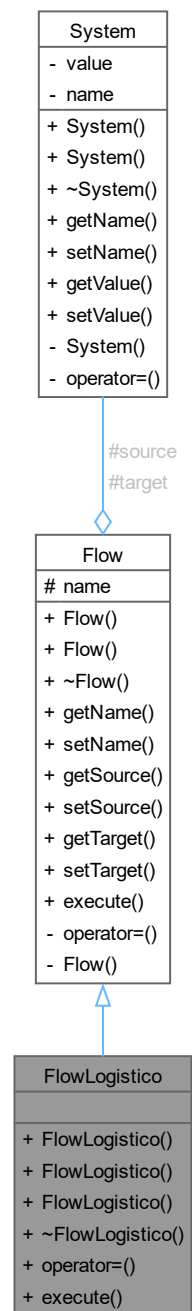
Classe responsável pelo fluxo logístico.

```
#include <flows.h>
```

Inheritance diagram for FlowLogistico:



Collaboration diagram for FlowLogistico:



Public Member Functions

- [FlowLogistico](#) ()
Construtor padrão da classe [FlowLogistico](#).
- [FlowLogistico](#) (const string &name, [System](#) *source, [System](#) *target)
Construtor parametrizado da classe [FlowLogistico](#).
- [FlowLogistico](#) (const [FlowLogistico](#) &other)

- Construtor de cópia da classe [FlowLogistico](#).*
- virtual [~FlowLogistico](#) ()
Destrutor virtual da classe [FlowLogistico](#).
- [FlowLogistico](#) & [operator=](#) (const [FlowLogistico](#) &other)
Operador de atribuição da classe [FlowLogistico](#).
- double [execute](#) () override
Executa o cálculo do fluxo logístico.

Public Member Functions inherited from [Flow](#)

- [Flow](#) ()
Construtor padrão da classe [Flow](#).
- [Flow](#) (const string &name, [System](#) *source, [System](#) *target)
Construtor parametrizado da classe [Flow](#).
- virtual [~Flow](#) ()
Destrutor virtual da classe [Flow](#).
- string [getName](#) () const
Retorna o nome do fluxo.
- void [setName](#) (const string &name)
Define o nome do fluxo.
- [System](#) * [getSource](#) () const
Retorna o sistema de origem.
- void [setSource](#) ([System](#) *source)
Define o sistema de origem.
- [System](#) * [getTarget](#) () const
Retorna o sistema de destino.
- void [setTarget](#) ([System](#) *target)
Define o sistema de destino.

Additional Inherited Members

Protected Attributes inherited from [Flow](#)

- string [name](#)
Nome do fluxo.
- [System](#) * [source](#)
Sistema de origem do fluxo.
- [System](#) * [target](#)
Sistema de destino do fluxo.

6.4.1 Detailed Description

Classe responsável pelo fluxo logístico.

Implementa um fluxo baseado em crescimento logístico entre sistemas.

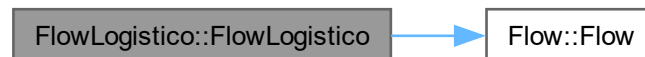
6.4.2 Constructor & Destructor Documentation

6.4.2.1 FlowLogistico() [1/3]

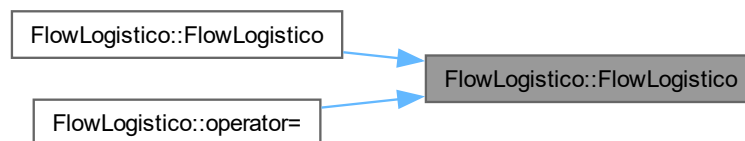
```
FlowLogistico::FlowLogistico ()
```

Construtor padrão da classe [FlowLogistico](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.4.2.2 FlowLogistico() [2/3]

```
FlowLogistico::FlowLogistico (
    const string & name,
    System * source,
    System * target)
```

Construtor parametrizado da classe [FlowLogistico](#).

Parameters

<i>name</i>	Nome do fluxo.
<i>source</i>	Sistema de origem.
<i>target</i>	Sistema de destino.

Here is the call graph for this function:



6.4.2.3 FlowLogistico() [3/3]

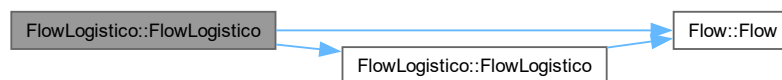
```
FlowLogistico::FlowLogistico (
    const FlowLogistico & other)
```

Construtor de cópia da classe [FlowLogistico](#).

Parameters

<i>other</i>	Outro objeto FlowLogistico .
--------------	--

Here is the call graph for this function:



6.4.2.4 ~FlowLogistico()

```
FlowLogistico::~~FlowLogistico () [virtual]
```

Destrutor virtual da classe [FlowLogistico](#).

Destrutor da classe [FlowLogistico](#).

6.4.3 Member Function Documentation

6.4.3.1 execute()

```
double FlowLogistico::execute () [override], [virtual]
```

Executa o cálculo do fluxo logístico.

Returns

Valor calculado pelo fluxo.

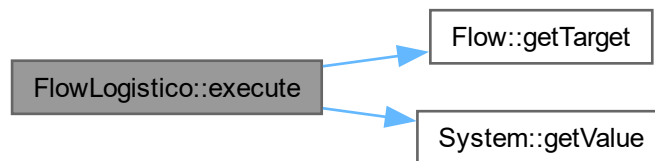
Calcula o crescimento logístico considerando a capacidade limite do sistema.

Returns

Valor calculado do fluxo.

Implements [Flow](#).

Here is the call graph for this function:

**6.4.3.2 operator=()**

```
FlowLogistico & FlowLogistico::operator= (
    const FlowLogistico & other)
```

Operador de atribuição da classe [FlowLogistico](#).

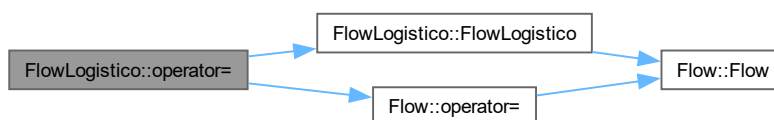
Parameters

<i>other</i>	Outro objeto FlowLogistico .
--------------	--

Returns

Referência para o objeto atual.

Here is the call graph for this function:



The documentation for this class was generated from the following files:

- `C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/include/flows.h`
- `C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/test/functional/flows.cpp`

6.5 Model Class Reference

Classe responsável por representar um modelo de simulação.

```
#include <model.h>
```

Collaboration diagram for Model:

Model
- systems
- flows
+ Model()
+ ~Model()
+ add()
+ add()
+ run()
+ showModel()
- Model()
- operator=()

Public Member Functions

- [Model](#) ()
Construtor padrão da classe [Model](#).
- virtual [~Model](#) ()
Destrutor virtual da classe [Model](#).
- void [add](#) ([System](#) *s)
Adiciona um sistema ao modelo.
- void [add](#) ([Flow](#) *f)
Adiciona um fluxo ao modelo.
- bool [run](#) (int t_init, int t_final)
Executa a simulação do modelo.
- void [showModel](#) () const
Exibe informações do modelo.

Private Member Functions

- [Model](#) (const [Model](#) &other)
Construtor de cópia privado.
- [Model](#) & [operator=](#) (const [Model](#) &other)
Operador de atribuição privado.

Private Attributes

- `vector< System * > systems`
Vetor contendo os sistemas do modelo.
- `vector< Flow * > flows`
Vetor contendo os fluxos do modelo.

6.5.1 Detailed Description

Classe responsável por representar um modelo de simulação.

A classe [Model](#) armazena os sistemas e fluxos que compõem um modelo de dinâmica de sistemas, sendo responsável pela execução da simulação.

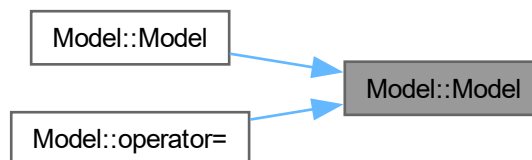
6.5.2 Constructor & Destructor Documentation

6.5.2.1 `Model()` [1/2]

```
Model::Model ()
```

Construtor padrão da classe [Model](#).

Here is the caller graph for this function:



6.5.2.2 `~Model()`

```
Model::~~Model () [virtual]
```

Destrutor virtual da classe [Model](#).

Destrutor da classe [Model](#).

Remove os elementos armazenados nos vetores de sistemas e fluxos.

6.5.2.3 Model() [2/2]

```
Model::Model (
    const Model & other) [private]
```

Construtor de cópia privado.

Construtor de cópia da classe [Model](#).

Impede cópia da classe [Model](#).

Parameters

<i>other</i>	Outro objeto Model .
<i>other</i>	Outro objeto Model .

Here is the call graph for this function:



6.5.3 Member Function Documentation

6.5.3.1 add() [1/2]

```
void Model::add (
    Flow * f)
```

Adiciona um fluxo ao modelo.

Parameters

<i>f</i>	Ponteiro para o fluxo a ser adicionado.
<i>f</i>	Ponteiro para o fluxo.

6.5.3.2 add() [2/2]

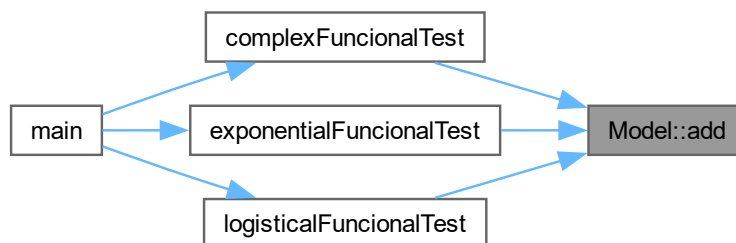
```
void Model::add (  
    System * s)
```

Adiciona um sistema ao modelo.

Parameters

s	Ponteiro para o sistema a ser adicionado.
s	Ponteiro para o sistema.

Here is the caller graph for this function:



6.5.3.3 operator=()

```
Model & Model::operator= (  
    const Model & other) [private]
```

Operador de atribuição privado.

Operador de atribuição da classe `Model`.

Impede atribuição entre objetos `Model`.

Parameters

other	Outro objeto <code>Model</code> .
-------	-----------------------------------

Returns

Referência para o próprio objeto.

Copia os sistemas e fluxos de outro modelo.

Parameters

<i>other</i>	Outro objeto Model .
--------------	--------------------------------------

Returns

Referência para o objeto atual.

Here is the call graph for this function:

**6.5.3.4 run()**

```

bool Model::run (
    int t_init,
    int t_final)
  
```

Executa a simulação do modelo.

Parameters

<i>t_init</i>	Tempo inicial da simulação.
<i>t_final</i>	Tempo final da simulação.

Returns

true caso a execução seja concluída com sucesso.

false caso ocorra falha na execução.

Percorre o intervalo de tempo especificado, executando os fluxos e atualizando os valores dos sistemas envolvidos.

Parameters

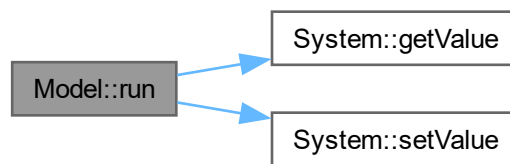
<i>t_init</i>	Tempo inicial da simulação.
---------------	-----------------------------

<code>t_final</code>	Tempo final da simulação.
----------------------	---------------------------

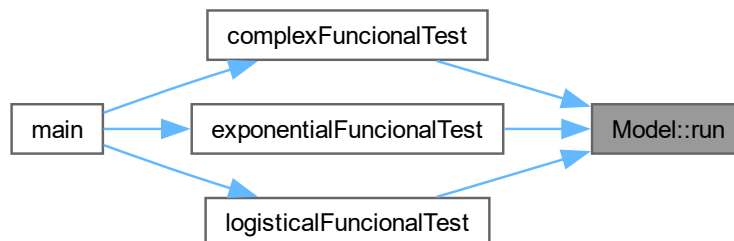
Returns

true caso a execução ocorra corretamente.

Here is the call graph for this function:



Here is the caller graph for this function:



6.5.3.5 showModel()

```
void Model::showModel () const
```

Exibe informações do modelo.

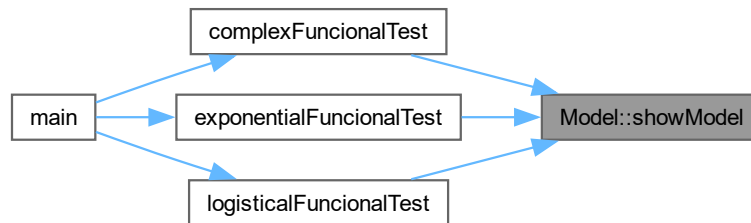
Exibe os sistemas e fluxos do modelo.

Mostra no terminal:

- sistemas cadastrados
- valores atuais

- conexões entre fluxos

Here is the caller graph for this function:



6.5.4 Member Data Documentation

6.5.4.1 flows

```
vector<Flow*> Model::flows [private]
```

Vetor contendo os fluxos do modelo.

6.5.4.2 systems

```
vector<System*> Model::systems [private]
```

Vetor contendo os sistemas do modelo.

The documentation for this class was generated from the following files:

- C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/include/[model.h](#)
- C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/src/[model.cpp](#)

6.6 System Class Reference

Classe responsável por representar um sistema.

```
#include <system.h>
```

Collaboration diagram for System:

System
- value
- name
+ System()
+ System()
+ ~System()
+ getName()
+ setName()
+ getValue()
+ setValue()
- System()
- operator=()

Public Member Functions

- [System](#) ()
Construtor padrão da classe [System](#).
- [System](#) (const string &name, double value)
Construtor da classe [System](#).
- virtual [~System](#) ()
Destrutor virtual da classe [System](#).
- string [getName](#) () const
Retorna o nome do sistema.
- void [setName](#) (const string &name)
Define o nome do sistema.
- double [getValue](#) () const
Retorna o valor atual do sistema.
- void [setValue](#) (double value)
Define o valor do sistema.

Private Member Functions

- [System](#) (const [System](#) &other)
Construtor de cópia privado.
- [System](#) & [operator=](#) (const [System](#) &other)
Operador de atribuição privado.

Private Attributes

- double [value](#)
Valor armazenado pelo sistema.
- string [name](#)
Nome do sistema.

6.6.1 Detailed Description

Classe responsável por representar um sistema.

Um sistema armazena um valor que pode ser alterado durante a execução da simulação.

6.6.2 Constructor & Destructor Documentation

6.6.2.1 `System()` [1/3]

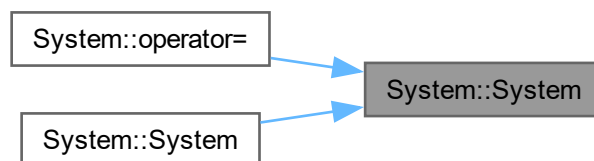
```
System::System ()
```

Construtor padrão da classe [System](#).

Inicializa o sistema com:

- valor igual a 0.0
- nome vazio

Here is the caller graph for this function:



6.6.2.2 System() [2/3]

```
System::System (
    const string & name,
    double value)
```

Construtor da classe [System](#).

Construtor parametrizado da classe [System](#).

Parameters

<i>name</i>	Nome do sistema.
<i>value</i>	Valor inicial do sistema.

6.6.2.3 ~System()

```
System::~~System () [virtual]
```

Destrutor virtual da classe [System](#).

Destrutor da classe [System](#).

6.6.2.4 System() [3/3]

```
System::System (
    const System & other) [private]
```

Construtor de cópia privado.

Construtor de cópia da classe [System](#).

Impede cópia da classe [System](#).

Parameters

<i>other</i>	Outro objeto System .
<i>other</i>	Outro objeto System .

Here is the call graph for this function:



6.6.3 Member Function Documentation

6.6.3.1 getName()

```
string System::getName () const
```

Retorna o nome do sistema.

Returns

Nome do sistema.

6.6.3.2 getValue()

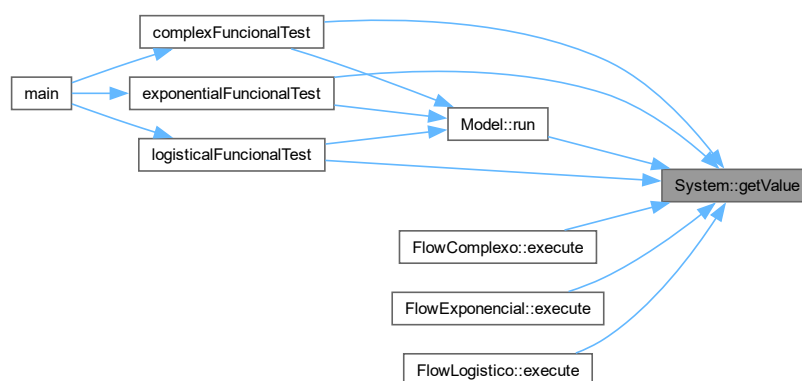
```
double System::getValue () const
```

Retorna o valor atual do sistema.

Returns

Valor armazenado no sistema.

Here is the caller graph for this function:



6.6.3.3 operator=()

```
System & System::operator= (
    const System & other) [private]
```

Operador de atribuição privado.

Operador de atribuição da classe [System](#).

Impede atribuição entre objetos [System](#).

Parameters

<i>other</i>	Outro objeto System .
--------------	---------------------------------------

Returns

Referência para o próprio objeto.

Copia os atributos de outro objeto [System](#).

Parameters

<i>other</i>	Outro objeto System .
--------------	---------------------------------------

Returns

Referência para o objeto atual.

Here is the call graph for this function:



6.6.3.4 setName()

```
void System::setName (
    const string & name)
```

Define o nome do sistema.

Parameters

<i>name</i>	Novo nome do sistema.
-------------	-----------------------

6.6.3.5 setValue()

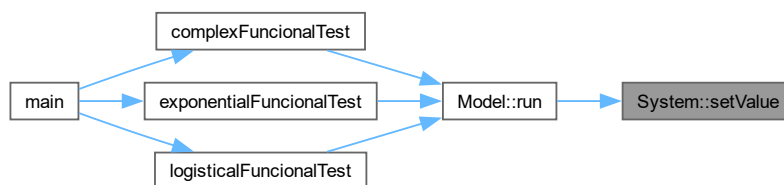
```
void System::setValue (
    double value)
```

Define o valor do sistema.

Parameters

<i>value</i>	Novo valor do sistema.
--------------	------------------------

Here is the caller graph for this function:

**6.6.4 Member Data Documentation****6.6.4.1 name**

```
string System::name [private]
```

Nome do sistema.

6.6.4.2 value

```
double System::value [private]
```

Valor armazenado pelo sistema.

The documentation for this class was generated from the following files:

- C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/include/[system.h](#)
- C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/src/[system.cpp](#)

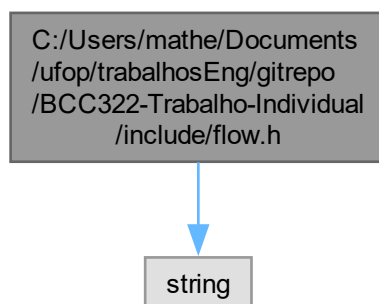
Chapter 7

File Documentation

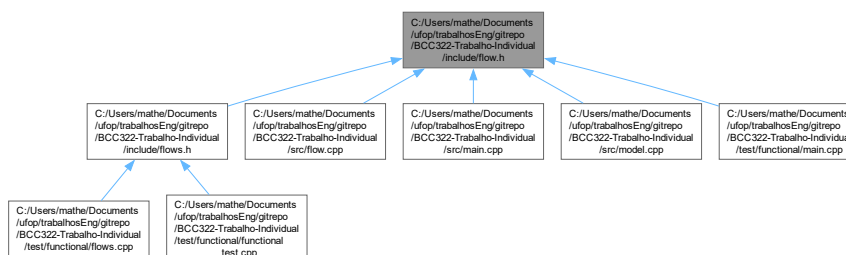
7.1 C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/include/flow.h File Reference

```
#include <string>
```

Include dependency graph for flow.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [Flow](#)

Classe abstrata responsável por representar um fluxo.

7.2 flow.h

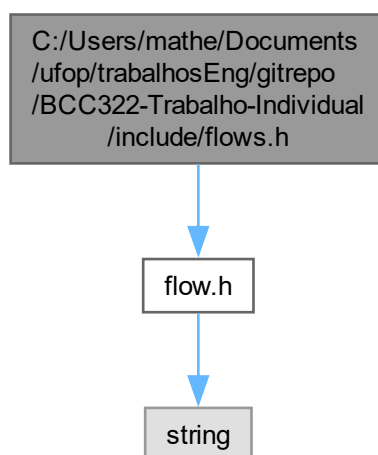
[Go to the documentation of this file.](#)

```
00001 #ifndef FLOW_H
00002 #define FLOW_H
00003
00004 #include <string>
00005
00006 using namespace std;
00007
00008 class System;
00009
00017 class Flow {
00018 protected:
00019
00023     string name;
00024
00028     System* source;
00029
00033     System* target;
00034
00035 public:
00036
00037     // Forma canônica
00038
00042     Flow();
00043
00051     Flow(const string& name,
00052          System* source,
00053          System* target);
00054
00058     virtual ~Flow();
00059
00060
00061     // Getters e setters
00062
00068     string getName() const;
00069
00075     void setName(const string& name);
00076
00082     System* getSource() const;
00083
00089     void setSource(System* source);
00090
00096     System* getTarget() const;
00097
00103     void setTarget(System* target);
00104
00105
00113     virtual double execute() = 0;
00114
00115 private:
00116
00125     Flow& operator=(const Flow& other);
00126
00134     Flow(const Flow& other);
00135 };
00136
00137 #endif
```

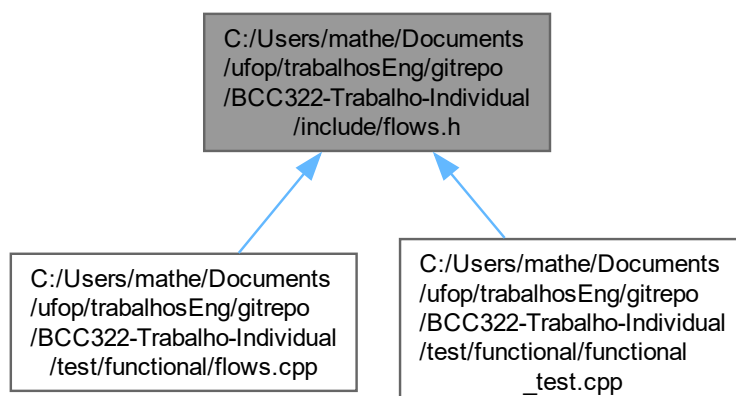
7.3 C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/include/flows.h File Reference

```
#include "flow.h"
```

Include dependency graph for flows.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [FlowExponencial](#)

Classe responsável pelo fluxo exponencial.

- class [FlowLogistico](#)

Classe responsável pelo fluxo logístico.

- class [FlowComplexo](#)

Classe responsável pelo fluxo complexo.

7.4 flows.h

[Go to the documentation of this file.](#)

```

00001 #ifndef FLOWS_H
00002 #define FLOWS_H
00003
00004 #include "flow.h"
00005
00012 class FlowExponencial : public Flow {
00013 public:
00014
00018     FlowExponencial();
00019
00027     FlowExponencial(
00028         const string& name,
00029         System* source,
00030         System* target
00031     );
00032
00038     FlowExponencial(
00039         const FlowExponencial& other
00040     );
00041
00045     virtual ~FlowExponencial();
00046
00053     FlowExponencial& operator=(
00054         const FlowExponencial& other
00055     );
00056
00062     double execute() override;
00063 };
00064
00065
00072 class FlowLogistico : public Flow {
00073 public:
00074
00078     FlowLogistico();
00079
00087     FlowLogistico(
00088         const string& name,
00089         System* source,
00090         System* target
00091     );
00092
00098     FlowLogistico(
00099         const FlowLogistico& other
00100     );
00101
00105     virtual ~FlowLogistico();
00106
00113     FlowLogistico& operator=(
00114         const FlowLogistico& other
00115     );
00116
00122     double execute() override;
00123 };
00124
00125
00132 class FlowComplexo : public Flow {
00133 public:
00134
00138     FlowComplexo();
00139
00147     FlowComplexo(
00148         const string& name,
00149         System* source,
00150         System* target
00151     );
00152
00158     FlowComplexo(
00159         const FlowComplexo& other
00160     );

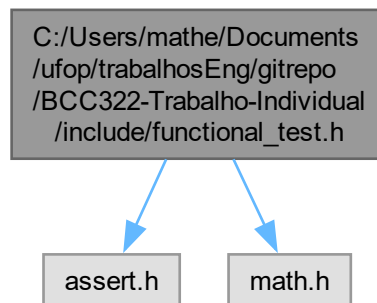
```



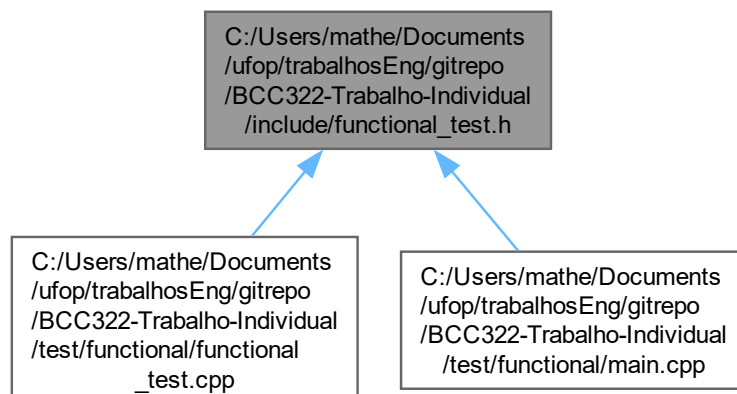
```
00161
00165     virtual ~FlowComplexo();
00166
00173     FlowComplexo& operator=(
00174         const FlowComplexo& other
00175     );
00176
00182     double execute() override;
00183 };
00184
00185 #endif
```

7.5 C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/include/functional_test.h File Reference

```
#include <assert.h>
#include <math.h>
Include dependency graph for functional_test.h:
```



This graph shows which files directly or indirectly include this file:



Functions

- void [exponentialFuncionalTest](#) ()
Executa o teste funcional do modelo exponencial.
- void [logisticalFuncionalTest](#) ()
Executa o teste funcional do modelo logístico.
- void [complexFuncionalTest](#) ()
Executa o teste funcional do modelo complexo.

7.5.1 Function Documentation

7.5.1.1 complexFuncionalTest()

```
void complexFuncionalTest ()
```

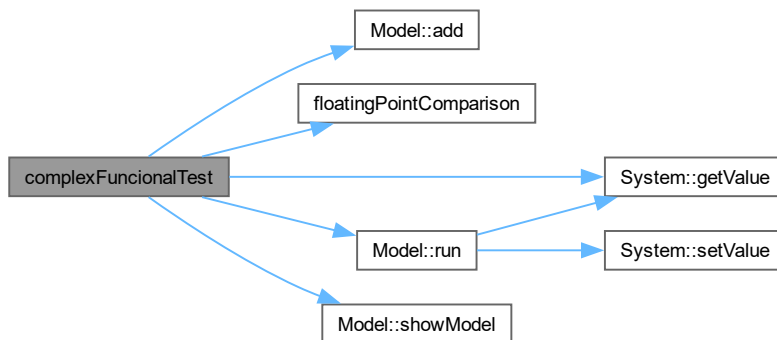
Executa o teste funcional do modelo complexo.

Executa o teste funcional complexo.

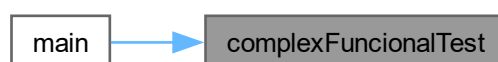
Verifica se o comportamento do modelo complexo produz os resultados esperados durante a simulação.

Executa o teste funcional do modelo complexo.

Verifica o comportamento do modelo complexo contendo múltiplos sistemas e fluxos. Here is the call graph for this function:



Here is the caller graph for this function:



7.5.1.2 exponentialFuncionalTest()

```
void exponentialFuncionalTest ()
```

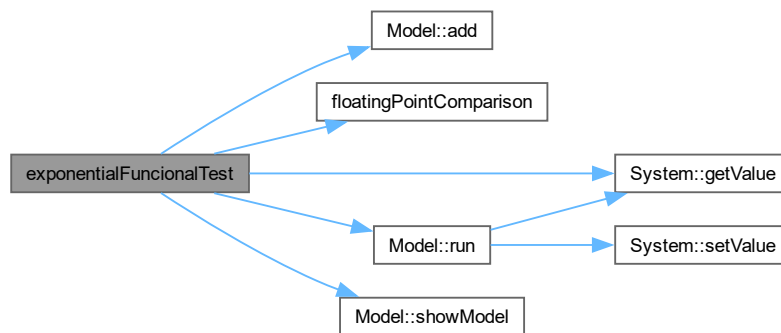
Executa o teste funcional do modelo exponencial.

Executa o teste funcional exponencial.

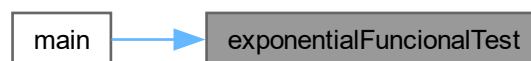
Verifica se o comportamento do fluxo exponencial produz os resultados esperados durante a simulação.

Executa o teste funcional do modelo exponencial.

Verifica o comportamento do modelo exponencial durante a simulação. Here is the call graph for this function:



Here is the caller graph for this function:



7.5.1.3 logisticalFuncionalTest()

```
void logisticalFuncionalTest ()
```

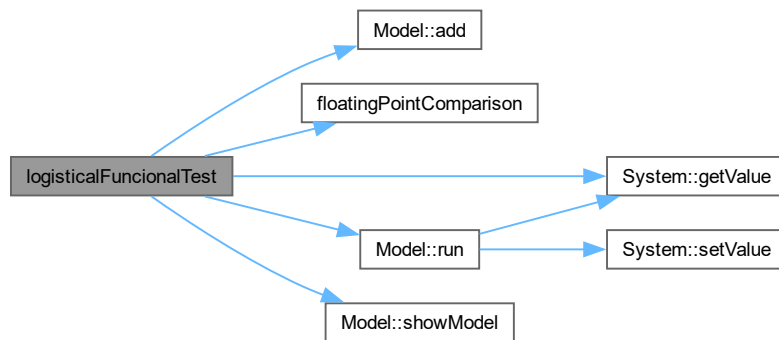
Executa o teste funcional do modelo logístico.

Executa o teste funcional logístico.

Verifica se o comportamento do fluxo logístico produz os resultados esperados durante a simulação.

Executa o teste funcional do modelo logístico.

Verifica o comportamento do modelo logístico durante a simulação. Here is the call graph for this function:



Here is the caller graph for this function:



7.6 functional_test.h

[Go to the documentation of this file.](#)

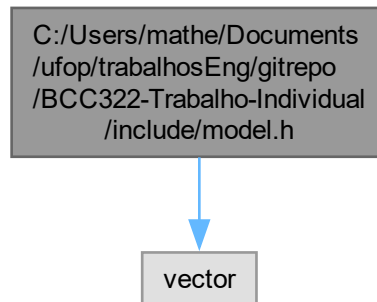
```

00001 #ifndef FUNCTIONAL_TESTS_H
00002 #define FUNCTIONAL_TESTS_H
00003
00004 #include <assert.h>
00005 #include <math.h>
00006
00013 void exponentialFuncionalTest();
00014
00021 void logisticalFuncionalTest();
00022
00029 void complexFuncionalTest();
00030
00031 #endif
  
```

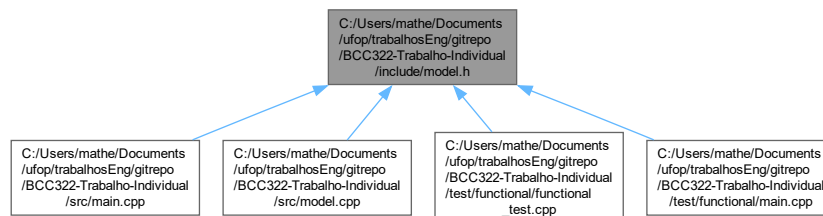
7.7 C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/include/model.h File Reference

```
#include <vector>
```

Include dependency graph for model.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [Model](#)

Classe responsável por representar um modelo de simulação.

7.8 model.h

[Go to the documentation of this file.](#)

```
00001 #ifndef MODEL_H
00002 #define MODEL_H
00003
00004 #include <vector>
00005
00006 using namespace std;
00007
00008 // Forward declaration
```

```

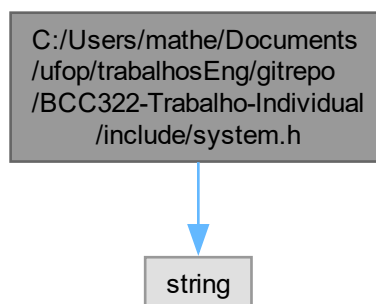
00009 class System;
00010 class Flow;
00011
00019 class Model {
00020 private:
00021
00025     vector<System*> systems;
00026
00030     vector<Flow*> flows;
00031
00032 public:
00033
00037     Model();
00038
00042     virtual ~Model();
00043
00044     // Métodos da UML
00045
00052     void add(System* s);
00053
00059     void add(Flow* f);
00060
00061
00071     bool run(int t_init, int t_final);
00072
00076     void showModel() const;
00077
00078 private:
00079
00087     Model(const Model& other);
00088
00097     Model& operator=(const Model& other);
00098 };
00099
00100 #endif

```

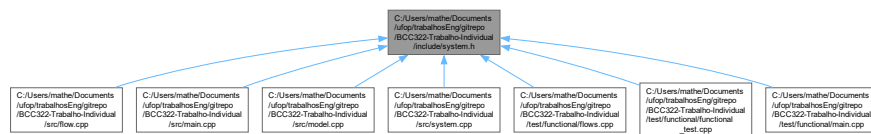
7.9 C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/include/system.h File Reference

#include <string>

Include dependency graph for system.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [System](#)

Classe responsável por representar um sistema.

7.10 system.h

[Go to the documentation of this file.](#)

```

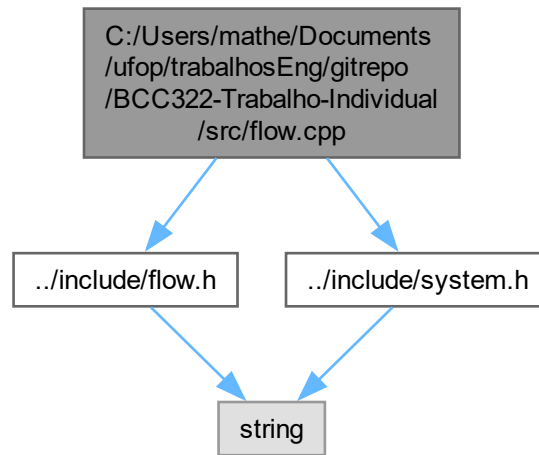
00001 #ifndef SYSTEM_H
00002 #define SYSTEM_H
00003
00004 #include <string>
00005
00006 using namespace std;
00007
00014 class System {
00015 private:
00016     double value;
00020
00021     string name;
00026
00027 public:
00028     System();
00032
00033     System(const string& name, double value);
00040
00041     virtual ~System();
00045
00046     // Métodos da UML
00047
00054     string getName() const;
00055
00061     void setName(const string& name);
00062
00068     double getValue() const;
00069
00075     void setValue(double value);
00076
00077 private:
00078     System(const System& other);
00087
00096     System& operator=(const System& other);
00097 };
00098
00099 #endif

```

7.11 C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/src/flow.cpp File Reference

Implementação da classe abstrata [Flow](#).

```
#include "../include/flow.h"
#include "../include/system.h"
Include dependency graph for flow.cpp:
```



7.11.1 Detailed Description

Implementação da classe abstrata [Flow](#).

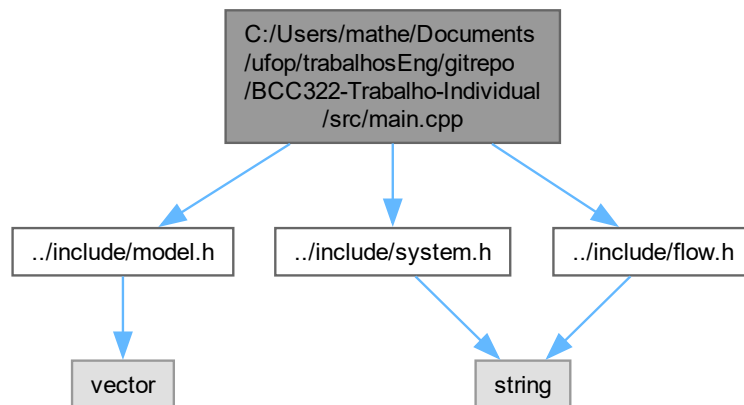
Este arquivo contém os métodos responsáveis pela manipulação dos fluxos do simulador.

7.12 C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/src/main.cpp File Reference

Arquivo principal do simulador.

```
#include "../include/model.h"
#include "../include/system.h"
#include "../include/flow.h"
```


Include dependency graph for main.cpp:



Macros

- `#define` [MAIN](#)

Functions

- `int` [main](#) ()
Função principal do simulador.

7.12.1 Detailed Description

Arquivo principal do simulador.

Este arquivo contém a função principal responsável pela inicialização da aplicação.

7.12.2 Macro Definition Documentation

7.12.2.1 MAIN

```
#define MAIN
```

7.12.3 Function Documentation

7.12.3.1 main()

```
int main ()
```

Função principal do simulador.

Responsável por iniciar a execução do programa.

Returns

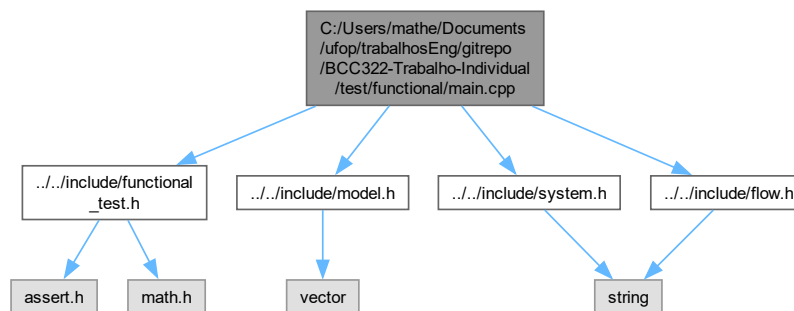
0 caso o programa execute corretamente.

7.13 C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/test/functional/main.cpp File Reference

Arquivo principal dos testes funcionais.

```
#include "../../include/functional_test.h"
#include "../../include/model.h"
#include "../../include/system.h"
#include "../../include/flow.h"
```

Include dependency graph for main.cpp:



Macros

- `#define` `MAIN_FUNCIONAL_TESTS`

Functions

- `int` `main` ()

Função principal dos testes funcionais.

7.13.1 Detailed Description

Arquivo principal dos testes funcionais.

Este arquivo executa todos os testes funcionais do simulador de sistemas dinâmicos.

7.13.2 Macro Definition Documentation

7.13.2.1 MAIN_FUNCIONAL_TESTS

```
#define MAIN_FUNCIONAL_TESTS
```

7.13.3 Function Documentation

7.13.3.1 main()

```
int main ()
```

Função principal dos testes funcionais.

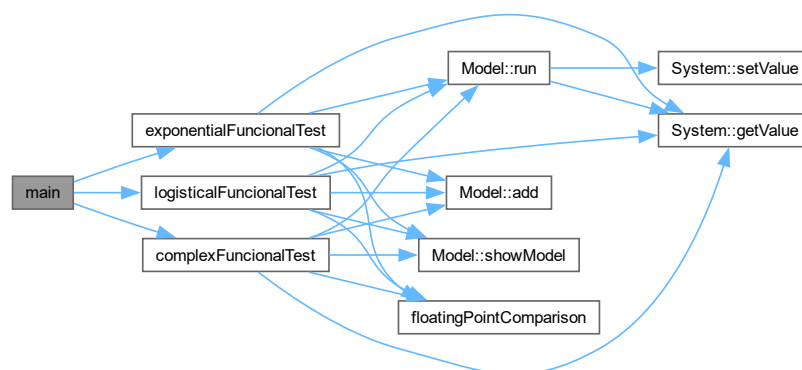
Executa:

- teste exponencial
- teste logístico
- teste complexo

Returns

0 caso os testes sejam executados com sucesso.

Here is the call graph for this function:



7.14 C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/test/unit/main.cpp File Reference

Functions

- int [main](#) ()

7.14.1 Function Documentation

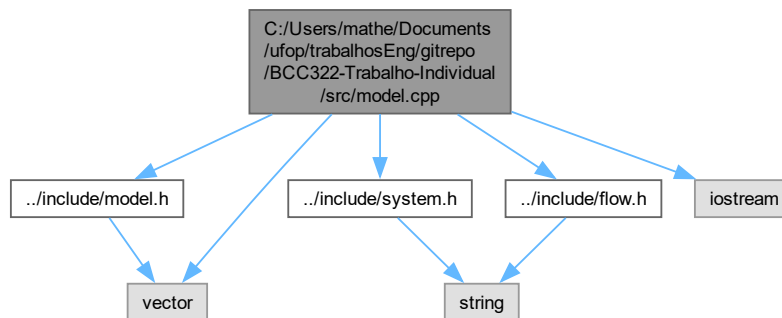
7.14.1.1 main()

```
int main ()
```

7.15 C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/src/model.cpp File Reference

Implementação da classe [Model](#).

```
#include "../include/model.h"
#include "../include/system.h"
#include "../include/flow.h"
#include <iostream>
#include <vector>
Include dependency graph for model.cpp:
```



7.15.1 Detailed Description

Implementação da classe [Model](#).

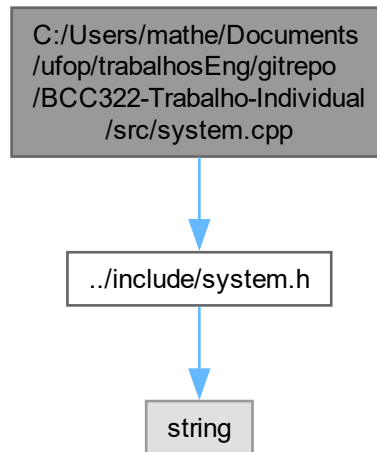
Este arquivo contém os métodos responsáveis pela manipulação e execução dos modelos de simulação do sistema dinâmico.

7.16 C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/src/system.cpp File Reference

Implementação da classe [System](#).

```
#include "../include/system.h"
```

Include dependency graph for system.cpp:



7.16.1 Detailed Description

Implementação da classe [System](#).

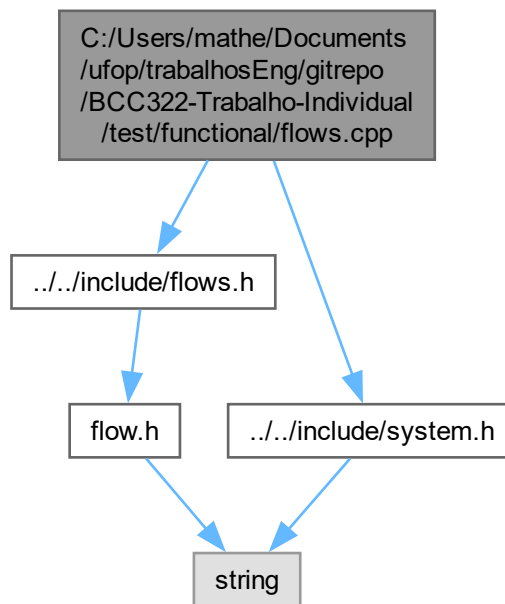
Este arquivo contém a implementação dos métodos responsáveis pela manipulação dos sistemas do simulador.

7.17 C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/test/functional/flows.cpp File Reference

Implementação dos fluxos do simulador.

```
#include "../../../include/flows.h"
#include "../../../include/system.h"
```

Include dependency graph for flows.cpp:



7.17.1 Detailed Description

Implementação dos fluxos do simulador.

Este arquivo contém as implementações dos fluxos:

- exponencial
- logístico
- complexo

7.18 C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/test/functional/functional_test.cpp File Reference

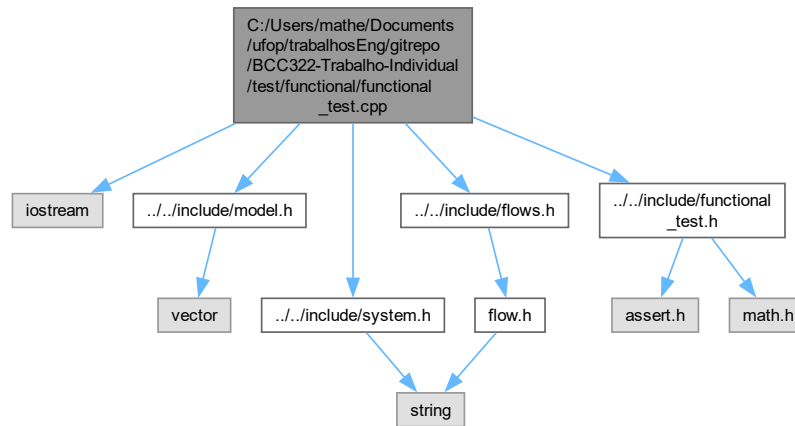
```

#include <iostream>
#include "../../include/model.h"
#include "../../include/system.h"
#include "../../include/flows.h"

```

```
#include "../..../include/functional_test.h"
```

Include dependency graph for functional_test.cpp:



Functions

- bool `floatingPointComparison` (double a, double b)
Realiza comparação entre números de ponto flutuante.
- void `exponentialFuncionalTest` ()
Executa o teste funcional exponencial.
- void `logisticalFuncionalTest` ()
Executa o teste funcional logístico.
- void `complexFuncionalTest` ()
Executa o teste funcional complexo.

7.18.1 Function Documentation

7.18.1.1 `complexFuncionalTest()`

```
void complexFuncionalTest ()
```

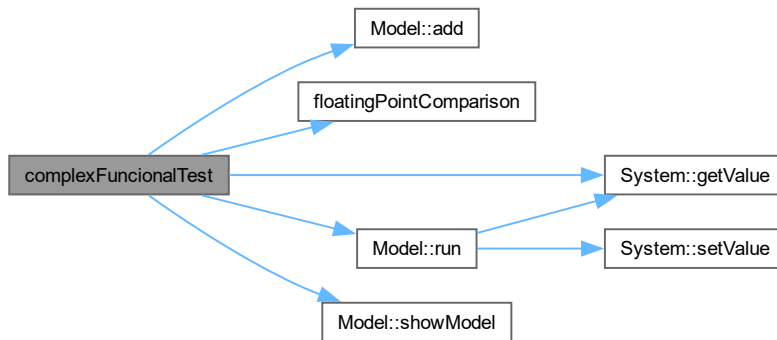
Executa o teste funcional complexo.

Executa o teste funcional do modelo complexo.

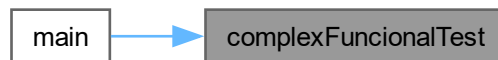
Verifica o comportamento do modelo complexo contendo múltiplos sistemas e fluxos.

Executa o teste funcional complexo.

Verifica se o comportamento do modelo complexo produz os resultados esperados durante a simulação. Here is the call graph for this function:



Here is the caller graph for this function:



7.18.1.2 exponentialFuncionalTest()

```
void exponentialFuncionalTest ()
```

Executa o teste funcional exponencial.

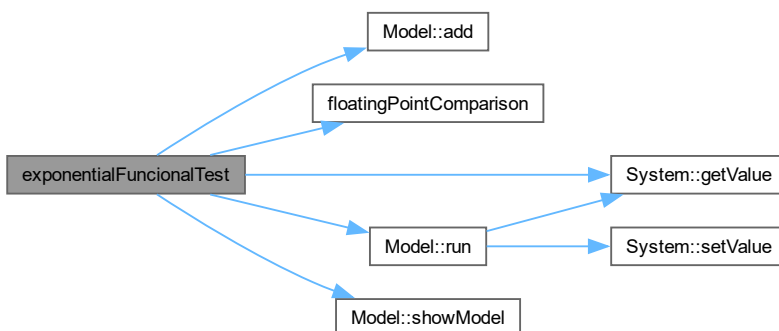
Executa o teste funcional do modelo exponencial.

Verifica o comportamento do modelo exponencial durante a simulação.

Executa o teste funcional exponencial.

Verifica se o comportamento do fluxo exponencial produz os resultados esperados durante a simulação. Here is

the call graph for this function:



Here is the caller graph for this function:



7.18.1.3 floatingPointComparison()

```
bool floatingPointComparison (
    double a,
    double b)
```

Realiza comparação entre números de ponto flutuante.

Utiliza uma margem de erro para evitar problemas de precisão numérica.

Parameters

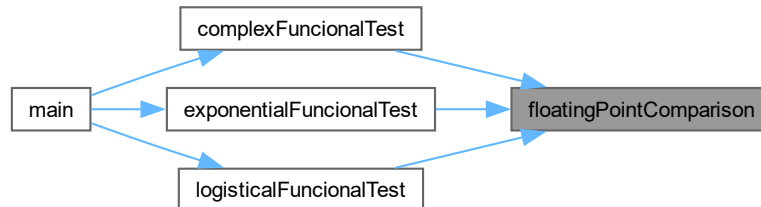
<i>a</i>	Primeiro valor.
<i>b</i>	Segundo valor.

Returns

true caso os valores sejam aproximadamente iguais.

false caso os valores sejam diferentes.

Here is the caller graph for this function:

**7.18.1.4 logisticalFuncionalTest()**

```
void logisticalFuncionalTest ()
```

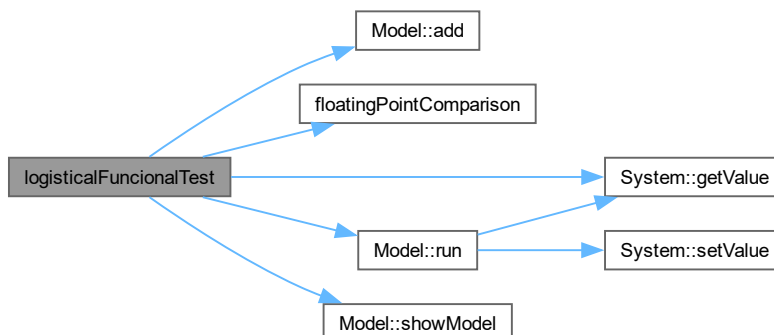
Executa o teste funcional logístico.

Executa o teste funcional do modelo logístico.

Verifica o comportamento do modelo logístico durante a simulação.

Executa o teste funcional logístico.

Verifica se o comportamento do fluxo logístico produz os resultados esperados durante a simulação. Here is the call graph for this function:



7.19

C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-Trabalho-Individual/test/unit/unit_test.cpp

File Reference

77

Here is the caller graph for this function:



7.19 C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-
Trabalho-Individual/test/unit/unit_test.cpp File Reference

Index

~Flow
 Flow, 18
~FlowComplexo
 FlowComplexo, 28
~FlowExponencial
 FlowExponencial, 34
~FlowLogistico
 FlowLogistico, 40
~Model
 Model, 43
~System
 System, 51
add
 Model, 44
C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-
 Trabalho-Individual/include Directory Reference, 10
C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-
 Trabalho-Individual/include/flow.h, 55, 56
C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-
 Trabalho-Individual/include/flows.h, 57, 58
C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-
 Trabalho-Individual/include/functional_test.h, 59, 62
C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-
 Trabalho-Individual/include/model.h, 63
C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-
 Trabalho-Individual/include/system.h, 64, 65
C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-
 Trabalho-Individual/src Directory Reference, 10
C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-
 Trabalho-Individual/src/flow.cpp, 65
C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-
 Trabalho-Individual/src/main.cpp, 66
C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-
 Trabalho-Individual/src/model.cpp, 70
C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-
 Trabalho-Individual/src/system.cpp, 70
C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-
 Trabalho-Individual/test Directory Reference, 11
C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-
 Trabalho-Individual/test/functional Directory Reference, 9
C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-
 Trabalho-Individual/test/functional/flows.cpp, 71
C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-
 Trabalho-Individual/test/functional/functional_test.cpp, 72
C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-
 Trabalho-Individual/test/functional/main.cpp, 68
C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-
 Trabalho-Individual/test/unit Directory Reference, 11
C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-
 Trabalho-Individual/test/unit/main.cpp, 69
C:/Users/mathe/Documents/ufop/trabalhosEng/gitrepo/BCC322-
 Trabalho-Individual/test/unit/unit_test.cpp, 77
complexFuncionalTest
 functional_test.cpp, 73
 functional_test.h, 60
execute
 Flow, 19
FlowComplexo, 28
FlowExponencial, 34
FlowLogistico, 40
exponentialFuncionalTest
 functional_test.cpp, 74
 functional_test.h, 60
EngPointComparison
 functional_test.cpp, 75
Eng, 13
~Flow, 18
execute, 19
Flow, 17, 18
getName, 19
getSource, 19
getTarget, 20
name, 22
operator=, 20
setName, 21
setSource, 22
setTarget, 22
source, 22
target, 22
FlowComplexo, 23
~FlowComplexo, 28
FlowComplexo, 27, 28
operator=, 29
Exponencial, 30
~FlowExponencial, 34
execute, 34

- FlowExponencial, [33](#), [34](#)
 - operator=, [35](#)
- FlowLogistico, [36](#)
 - ~FlowLogistico, [40](#)
 - execute, [40](#)
 - FlowLogistico, [39](#), [40](#)
 - operator=, [41](#)
- flows
 - Model, [48](#)
- functional_test.cpp
 - complexFuncionalTest, [73](#)
 - exponentialFuncionalTest, [74](#)
 - floatingPointComparison, [75](#)
 - logisticalFuncionalTest, [76](#)
- functional_test.h
 - complexFuncionalTest, [60](#)
 - exponentialFuncionalTest, [60](#)
 - logisticalFuncionalTest, [61](#)
- getName
 - Flow, [19](#)
 - System, [52](#)
- getSource
 - Flow, [19](#)
- getTarget
 - Flow, [20](#)
- getValue
 - System, [52](#)
- logisticalFuncionalTest
 - functional_test.cpp, [76](#)
 - functional_test.h, [61](#)
- MAIN
 - main.cpp, [67](#)
- main
 - main.cpp, [67](#), [69](#)
- main.cpp
 - MAIN, [67](#)
 - main, [67](#), [69](#)
 - MAIN_FUNCIONAL_TESTS, [68](#)
- MAIN_FUNCIONAL_TESTS
 - main.cpp, [68](#)
- Model, [42](#)
 - ~Model, [43](#)
 - add, [44](#)
 - flows, [48](#)
 - Model, [43](#)
 - operator=, [45](#)
 - run, [46](#)
 - showModel, [47](#)
 - systems, [48](#)
- name
 - Flow, [22](#)
 - System, [54](#)
- operator=
 - Flow, [20](#)
- FlowComplejo, [29](#)
 - FlowExponencial, [35](#)
 - FlowLogistico, [41](#)
 - Model, [45](#)
 - System, [52](#)
- run
 - Model, [46](#)
- setName
 - Flow, [21](#)
 - System, [53](#)
- setSource
 - Flow, [22](#)
- setTarget
 - Flow, [22](#)
- setValue
 - System, [54](#)
- showModel
 - Model, [47](#)
- source
 - Flow, [22](#)
- System, [48](#)
 - ~System, [51](#)
 - getName, [52](#)
 - getValue, [52](#)
 - name, [54](#)
 - operator=, [52](#)
 - setName, [53](#)
 - setValue, [54](#)
 - System, [50](#), [51](#)
 - value, [54](#)
- systems
 - Model, [48](#)
- target
 - Flow, [22](#)
- value
 - System, [54](#)